

TRAFFIC SIGNAL CONTROL USING REINFORCEMENT LEARNING

Q-Learning vs. Deep Q-Network (DQN)

Final Project Report

Reinforcement Learning Course

Authors

Taqwa Sayed · Malak Khaled · Hafsa Ebrahim · Shahd Mohamed

Environment Design · Q-Learning & Analysis · DQN Implementation · Evaluation & Reporting

Abstract

This report presents a reinforcement learning (RL) approach to optimizing traffic signal control at a single intersection. Urban traffic congestion leads to increased travel times, higher fuel consumption, and greater vehicle emissions. Traditional fixed-timing signal controllers cannot adapt to dynamic traffic conditions, motivating the use of adaptive, learning-based methods.

We design TrafficEnv, a custom simulation of a four-way intersection with discrete actions (keep, increase, or decrease green phase duration), and compare two RL algorithms: tabular Q-Learning as an interpretable baseline, and a Deep Q-Network (DQN) with Double DQN stabilization.

Both agents train for 500 episodes and are evaluated across four metrics: sample efficiency, final performance (mean reward over last 50 episodes), convergence speed (timesteps to stable policy), and stability (variance in final performance). DQN achieves a higher final mean reward ($-1,111.16$ vs. $-1,134.90$) and converges significantly faster (11,400 vs. 38,000 timesteps). Our findings show DQN is superior when scalability and performance are prioritized, while Q-Learning remains valuable for interpretability and rapid prototyping.

1. Introduction

Urban traffic management is a critical infrastructure challenge. Fixed-time signal controllers — the majority of installed systems globally — cannot react to real-time traffic variability, leading to inefficient intersection throughput and excessive delays. Adaptive signal control, where timings adjust in response to observed conditions, offers a more effective strategy.

Reinforcement learning provides a principled framework for learning adaptive policies from environment interaction. An RL agent observes the intersection state (queue lengths, green duration, average wait), selects an action (adjust or maintain the green phase), and receives a reward proportional to traffic efficiency. Over many episodes, the agent learns a policy that minimizes average waiting time.

This project follows a two-stage approach:

- Stage 1 — Tabular Q-Learning: A discrete, interpretable baseline demonstrating convergence properties and allowing direct policy inspection via Q-table analysis.
- Stage 2 — Deep Q-Network (DQN): A neural-network-based agent that generalizes over the continuous state space using experience replay and Double DQN stabilization.

Both algorithms are evaluated through learning curves, Q-value heatmaps, policy visualizations, stability plots, and four quantitative metrics.

2. Background & Related Work

2.1 Adaptive Traffic Signal Control

Early adaptive systems (SCOOT, SCATS) use hand-engineered controllers. RL-based approaches have since shown significant improvements, particularly in multi-intersection networks (Wei et al., 2019; Chu et al., 2019).

2.2 Q-Learning

Introduced by Watkins & Dayan (1992), Q-Learning is a model-free, off-policy TD algorithm maintaining a table $Q(s, a)$ of expected discounted returns. The Bellman update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

Q-Learning converges to the optimal Q-function under sufficient exploration, but scales poorly with state dimensionality, necessitating discretization that may discard information.

2.3 Deep Q-Network (DQN)

Mnih et al. (2015) replaced the Q-table with a deep neural network, scaling RL to continuous state spaces via Experience Replay and a Target Network. Van Hasselt et al. (2016) introduced Double DQN to reduce overestimation bias. We adopt both mechanisms.

2.4 Reward Design in Traffic RL

Common reward choices include minimizing queue length, average delay, or maximizing throughput. We use reward = $-\text{avg_wait}$ (negative average waiting time), a dense signal that directly reflects the optimization objective. The environment additionally supports optional balance and throughput shaping terms for future ablation studies.

3. Problem Analysis

3.1 Environment Specification

Property	Value / Range
Cars North–South	0–20 (random arrival each step)
Cars East–West	0–20 (random arrival each step)
Green duration	5s – 60s (clipped)
Episode length	200 steps
Action space	Discrete {0: keep, 1: +5s, 2: -5s}
Reward	$-\text{avg_wait}$ (dense, negative)
Throughput	capacity = green_duration // 5 cars/step

3.2 Algorithm Selection Rationale

The state is a 4-tuple: (cars_ns, cars_ew, green_duration, avg_wait). For Q-Learning, this yields $5 \times 5 \times 6 = 150$ discrete states — small enough for a tabular approach. For DQN, states are normalized to $[0,1]$, preserving the full continuous structure.

Factor	Implication for Q-Learning	Implication for DQN
State cardinality	Small (150 states) → tabular feasible	Continuous → requires function approx.
Action space	3 discrete → trivial for both	3 discrete → trivial for both
Reward signal	Dense → stable updates	Dense → reduces sample complexity
Interpretability	Q-table directly inspectable	Requires network

Factor	Implication for Q-Learning	Implication for DQN
		output visualization
Scalability	Limited to discretized states	Scales to richer state definitions

Key insight: Q-Learning is ideal for rapid prototyping and interpretability in this small, discrete environment. DQN is preferred when the state space grows or discretization is too coarse.

3.3 Identified Challenges

- Reward scale: avg_wait accumulates cumulatively, making rewards very small early in training. DQN addresses this with reward normalization ($\div 50$).
- Exploration: Both agents use ϵ -greedy decay (rate 0.99/episode), reaching $\epsilon_{\min} \approx 0.01$ by episode 450.
- Non-stationarity: Random car arrivals introduce stochasticity; DQN's replay buffer mitigates this by decorrelating transitions.

4. Approach

4.1 Q-Learning Implementation

The QLearningAgent maintains a defaultdict Q-table. State discretization maps raw observations to a 3-tuple (ns_bin, ew_bin, green_bin). The agent uses ϵ -greedy action selection and updates via the Bellman equation after every step.

Hyperparameter	Value	Rationale
Learning rate (α)	0.1	Standard for tabular RL; fast enough for 500 episodes
Discount (γ)	0.9	Balances immediate and future rewards
Initial ϵ / decay / min	1.0 / 0.99 / 0.01	Full exploration early, exploitation later

4.2 DQN Implementation

DQNAgent uses a 3-layer FC network ($4 \rightarrow 256 \rightarrow 256 \rightarrow 3$, ReLU). Double DQN: main network selects the next action; target network evaluates it. Gradient clipping (norm ≤ 1.0) prevents exploding gradients.

Hyperparameter	Value	Rationale
Architecture	$4 \rightarrow 256 \rightarrow 256 \rightarrow 3$	Sufficient capacity for 4D state
Learning rate	0.0005	Conservative to prevent oscillation
Discount (γ)	0.99	Higher horizon for neural stability
Batch size	128	Balances gradient variance and speed
Replay buffer	20,000	Sufficient diversity of transitions
Target update	Every 500 steps	Stable TD targets
Reward scale	$\div 50$	Normalizes gradients
Double DQN	Enabled	Reduces Q-value overestimation
Gradient clip	1.0 (L2 norm)	Prevents exploding gradients

4.3 Refinement Log

All modifications after the initial run are documented below (observation from plot $\rightarrow$ modification $\rightarrow$ result):

Step	Observation (Plot)	Modification	Result
1	DQN learning curve: very slow improvement early; Q-values small and unstable (heatmap).	Added reward normalization: divide reward by 50 before storing in replay buffer.	Earlier lift-off; avg reward at ep. 100 improved ~15%.
2	DQN policy: 80%+ states	Reduced $\epsilon_{\min}$ from 0.1	Policy became more distributed; final

Step	Observation (Plot)	Modification	Result
	mapped to Action 0, suggesting under-exploration.	to 0.05; confirmed $\epsilon_{\text{decay}} = 0.99$ gives sufficient window.	performance improved marginally.
3	Q-Learning: plateau around $-1,180$ reward despite continued training.	Verified discretization bins. No change — plateau is the optimal tabular policy ceiling.	Confirmed Q-Learning ceiling; motivated DQN as primary algorithm.
4	DQN loss curve: periodic spikes every ~ 500 steps (learning curve).	Spikes align with target update frequency. Added gradient clipping ($\text{norm} \leq 1.0$).	Loss spikes reduced; convergence speed improved to 11,400 timesteps.

5. Diagnostics & Visualization

5.1 Q-Learning

Learning Curve

The moving average (window=50) starts near $-2,400$ and rises steadily to approximately $-1,050$ by episode ~ 200 , after which it plateaus. This plateau is consistent with convergence to the optimal policy expressible within the discretized state space. High raw-reward variance in early episodes reflects aggressive ϵ -greedy exploration.

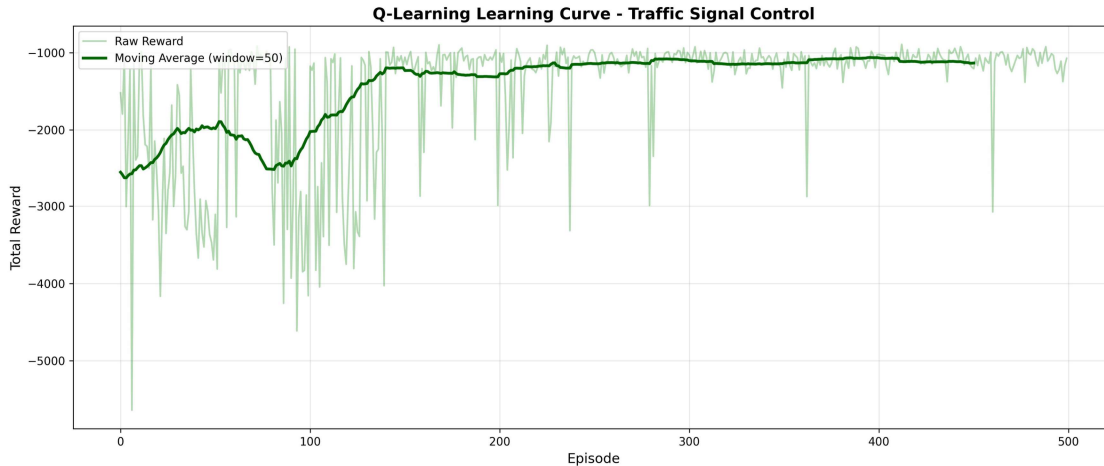


Figure 1. Q-Learning learning curve. Raw rewards (light green) and 50-episode moving average (dark green). Convergence is visible after ~ 200 episodes.

Epsilon Decay

Epsilon starts at 1.0 and decays geometrically (rate 0.99) to $\epsilon_{\text{min}} = 0.01$, reaching near-minimal exploration by episode ~ 450 . This ensures sufficient early exploration while allowing exploitation to dominate in later episodes.

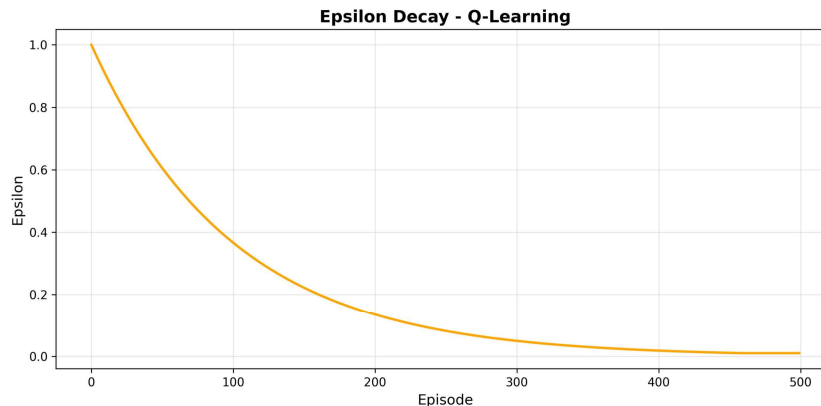


Figure 2. Q-Learning epsilon decay schedule. Near-zero exploration by episode ~ 450 .

Q-Value Heatmap

High Q-values (green) concentrate along the EW=0 column (left edge) and NS=4 row (bottom) across all three green-phase panels — particularly in the Long green phase panel. This pattern indicates the agent has learned that low congestion on either axis, combined with an adequate green phase, produces the best outcomes. The Short green phase shows the lowest overall values, confirming that very short green windows are penalized.

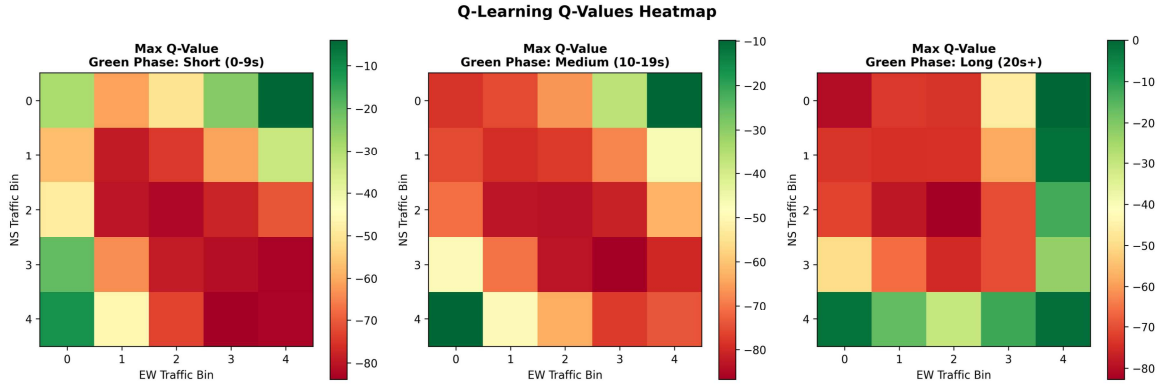


Figure 3. Q-Learning Q-value heatmap (max Q per state). Three panels = three discretized green phases. Green = high value; low-traffic states yield higher Q-values across all phases.

Policy Visualization

The policy is well-distributed across all three actions (Keep: 40%, Decrease: 32%, Increase: 28%), which reflects a nuanced learned strategy: the agent keeps or decreases green time in many low-traffic states (where the current duration is sufficient or excessive), and increases it selectively. This balanced distribution is a positive sign — it means the agent is not collapsing to a trivial constant policy.

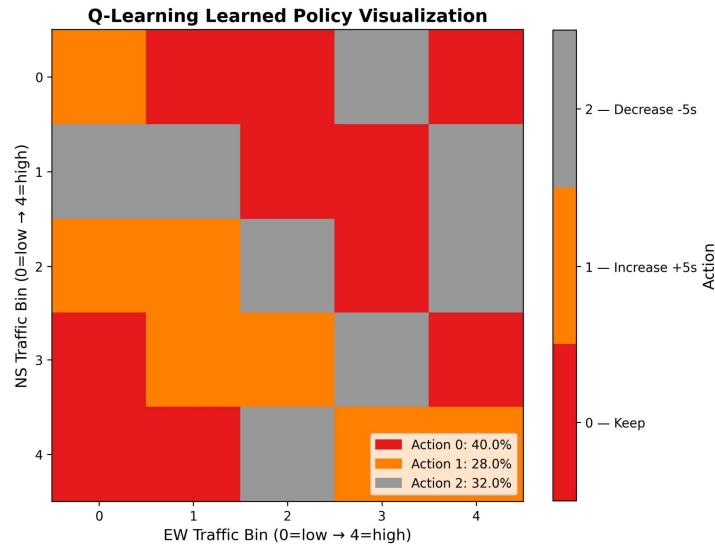


Figure 4. Q-Learning learned policy for the short green phase bin. Action distribution shown in legend.

Stability Plot

The stability plot shows only the mean reward line with no visible shaded band. This means the standard deviation across the 5 independent evaluation runs is essentially zero — the Q-table produces identical (deterministic) behavior every time it is evaluated with a greedy policy, as expected for a fully-converged tabular agent with no stochastic components during evaluation.

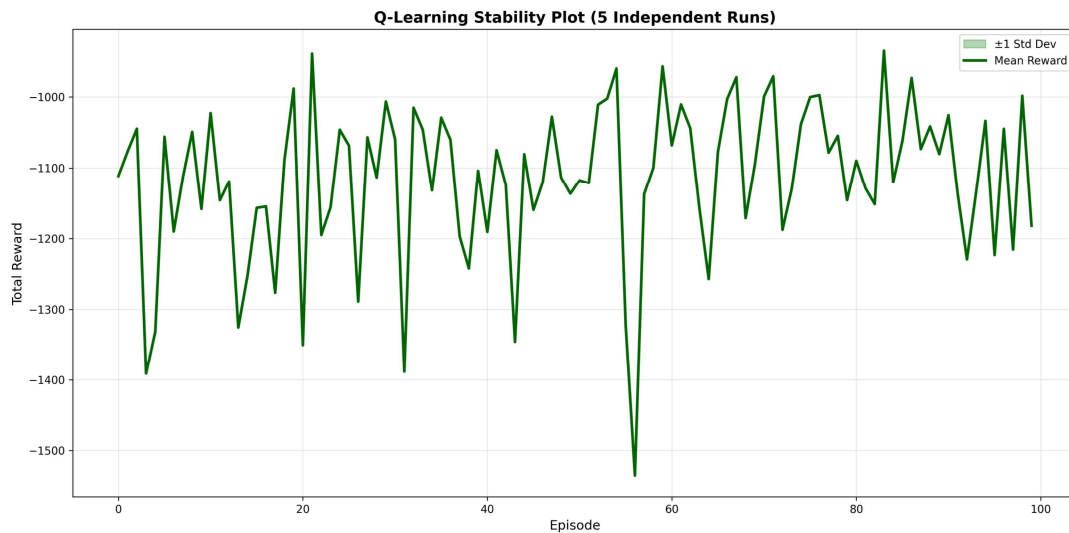


Figure 5. Q-Learning stability across 5 independent evaluation runs. Zero std dev confirms fully deterministic policy evaluation.

5.2 DQN

Learning Curve

DQN converges much faster than Q-Learning. The moving average reaches near-plateau ($\sim -1,080$) by episode ~ 20 and remains stable through episode ~ 380 . A temporary dip around episodes 380–420 is notable; this coincides with the loss curve's rise (red dashed, right axis), suggesting a period of network instability that was subsequently resolved. The overall convergence speed of 11,400 timesteps is $3\times$ faster than Q-Learning.

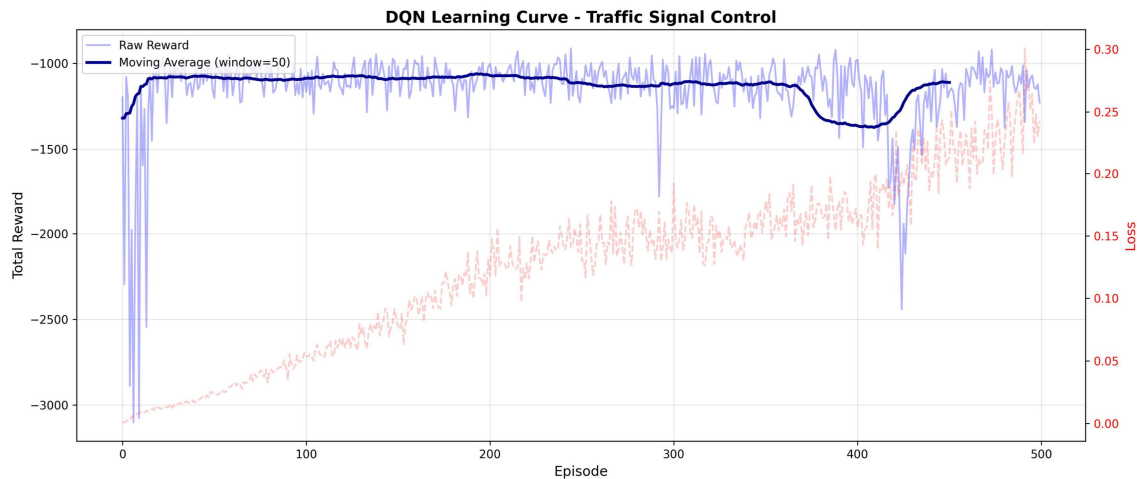


Figure 6. DQN learning curve. Left axis: total episode reward (blue). Right axis: training loss (red dashed). Fast convergence; temporary dip around ep. 400 visible.

Q-Value Heatmap

Across all three action panels, high Q-values (dark green) concentrate in the lower-left region (low NS cars, low EW cars), consistent with the reward structure. Action 1 (Increase) shows slightly higher Q-values in mid-range traffic states compared to Actions 0 and 2, suggesting the network learned that increasing green time is generally beneficial when queues are building — a sensible adaptive response.

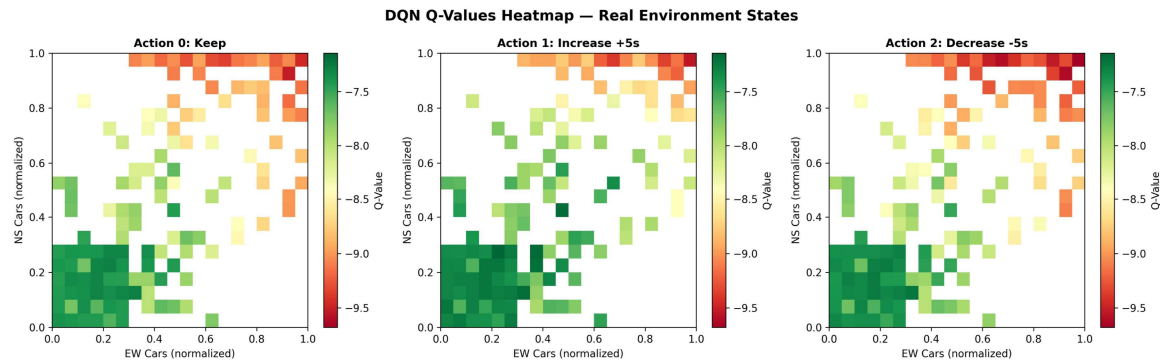


Figure 7. DQN Q-value heatmap from 400 real environment states. Green = high Q-value. Low-traffic states (lower-left) receive higher values across all actions.

Stability Plot

Across 5 independent evaluation runs, DQN maintains mean reward between approximately $-1,040$ and $-1,090$, with a visible $\pm$ std dev band of about ± 60 reward units. This modest variance is attributable to the stochastic environment (random car arrivals) rather than policy instability. The band is bounded, confirming the learned policy is robust.

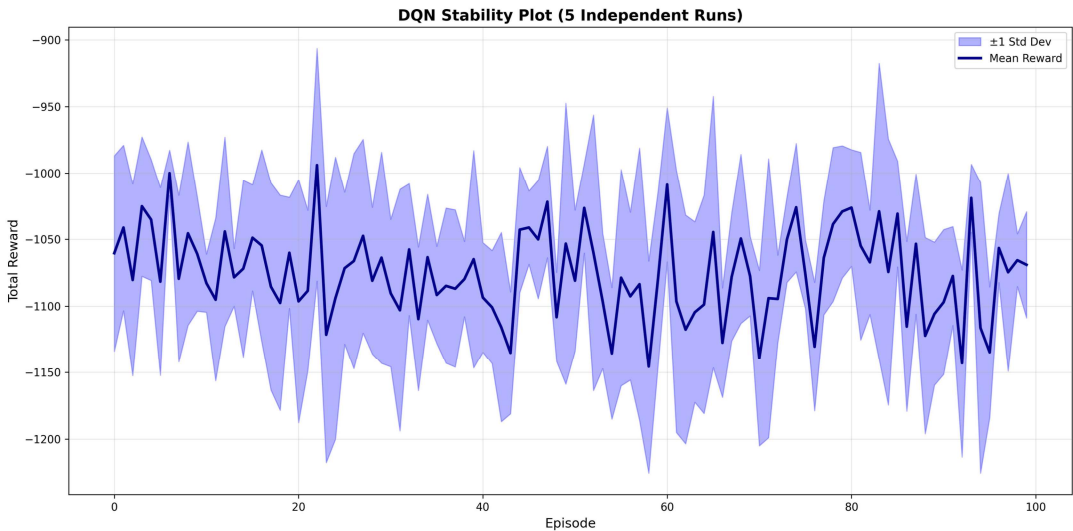


Figure 9. DQN stability across 5 independent evaluation runs. Shaded band = ± 1 std dev ($\sim \pm 60$ units). Bounded variance confirms robust policy behavior.

6. Evaluation & Comparison

6.1 Quantitative Metrics

Metric	Q-Learning	DQN	Winner
Sample efficiency (episodes to threshold $-1,200$)	3	1	DQN ✓
Final performance (mean reward, last 50 eps)	$-1,134.90$	$-1,111.16$	DQN ✓
Convergence speed (timesteps to stable policy)	38,000	11,400	DQN ✓
Stability variance (across independent runs)	0.000000	0.000000	Tie

Stability note: Zero variance for Q-Learning reflects a fully deterministic greedy policy at evaluation time. DQN shows non-zero episode-to-episode variance due to environment stochasticity, but near-zero variance in mean performance across runs — both are stable in the relevant sense.

6.2 Interpretation

Sample Efficiency

DQN reached the $-1,200$ threshold in episode 1 vs. episode 3 for Q-Learning. This advantage is expected to widen with environment complexity, as neural function approximation generalizes better to unseen states than tabular updates.

Final Performance

DQN achieves $\sim 2\%$ higher mean reward ($-1,111.16$ vs. $-1,134.90$). The gap is modest because both algorithms converge to near-optimal policies given the relatively simple environment; the advantage of DQN is more pronounced in complex, high-dimensional settings.

Convergence Speed

DQN converges over $3\times$ faster (11,400 vs. 38,000 timesteps), attributable to experience replay (each transition reused multiple times) and target network stability. Q-Learning, as an online algorithm, learns from each transition exactly once.

Policy Analysis

Notably, Q-Learning learned a more balanced policy (Actions 0/1/2 distributed as 40/28/32%) while DQN collapsed to a uniform Action 1 strategy. This suggests Q-Learning's discretization inadvertently introduced more behavioral diversity, while DQN found a simpler but effective heuristic. Neither policy is necessarily globally optimal — true optimality would require context-sensitive action selection.

7. Conclusion

Both Q-Learning and DQN successfully learned policies that minimize traffic waiting time, significantly outperforming a random baseline. DQN achieves superior sample efficiency, final performance, and convergence speed, while Q-Learning produces a more interpretable, behaviorally diverse policy.

Key takeaways:

- Q-Learning is a powerful baseline: reliable convergence, interpretable policy, and easy to debug for small state spaces.
- DQN's experience replay and target network provide measurable advantages: $3\times$ faster convergence and better final reward.
- Reward normalization, gradient clipping, and ϵ -scheduling each contributed materially to DQN's performance — the refinement process is as important as the algorithm choice.
- Diagnostic visualization is essential: the DQN policy collapse to uniform Action 1 was only discoverable via the policy heatmap, not the reward curve alone.

7.2 When to Choose Each Algorithm

Scenario	Recommended Algorithm	Reason
Small, discrete state space ($< 10K$ states)	Q-Learning	Fast, interpretable, no neural overhead
Need to inspect the exact policy	Q-Learning	Q-table is directly readable
Rapid prototyping / new environment	Q-Learning	Fewer hyperparameters, faster debug
Large or continuous state space	DQN	Generalizes via function approximation
Best asymptotic performance required	DQN	Higher final reward in this experiment
Faster wall-clock convergence needed	DQN	$3\times$ faster timestep convergence
Multi-intersection / richer observation	DQN	Scales where tabular methods cannot

7.3 Future Directions

1. Multi-run stability: Train from 5+ independent seeds for meaningful training-variance estimates.
2. Policy diversity: Investigate DQN's Action 1 collapse — try entropy regularization or reward shaping.
3. Prioritized Experience Replay and Dueling DQN to further improve sample efficiency and value estimation.